

## Softwarekonstruktion WS 15/16 – Übung 6

Ausgabe: 18.01.2016, Abgabe: 01.02.2016 12:00 Uhr

### Präsenzübung

#### 6.1 Signatur und Algebra

Gegeben Sei folgende Signatur:

```
 $\Sigma =$  sorts  int  
      opns  zero:  $\rightarrow$  int  
            succ: int  $\rightarrow$  int  
            inv:  int  $\rightarrow$  int
```

Geben Sie eine  $\Sigma$ -Algebra  $A$  an, so dass **zero**, **succ** und **inv** die generierenden Operationen der Menge der ganzen Zahlen sind.

Hinweis: Die Operation **inv** soll einen Vorzeichenwechsel für alle Zahlen außer 0 bewirken.

#### 6.2 Algebraische Spezifikation: Liste

Erstellen Sie die algebraische Spezifikation einer Liste mit folgenden Operationen:

1. **new**: Erzeugt eine leere Liste
2. **add**: Fügt ein Element an das Ende der Liste an
3. **del**: Löscht ein Element am Ende der Liste
4. **con**: Fügt zwei Listen aneinander
5. **head**: Liefert das erste Element der Liste
6. **tail**: Liefert das letzte Element der Liste
7. **len**: Liefert die Länge der Liste

Die Elemente der Liste sind vom abstrakten Datentyp **elem**.

## Hausübung

### 6.3 Signatur und Algebra: Reelle Zahlen (2 Punkte)

Gegeben sei folgende Signatur der reellen Zahlen:

```
 $\Sigma =$  sorts real  
      opns zero:  $\rightarrow$  real  
           one:  $\rightarrow$  real  
           add:  $\text{real} \times \text{real} \rightarrow \text{real}$   
           sub:  $\text{real} \times \text{real} \rightarrow \text{real}$   
           mult:  $\text{real} \times \text{real} \rightarrow \text{real}$   
           div:  $\text{real} \times \text{real} \rightarrow \text{real}$ 
```

Geben Sie eine Algebra an, die die Signatur interpretiert.

### 6.4 Algebraische Spezifikation: Stack (5 Punkte)

Geben Sie eine algebraische Spezifikation für eine Stack-Datenstruktur mit den folgenden Operationen an:

1. **empty** : erzeugt einen leeren Stack
2. **push** : legt ein Element auf den Stack
3. **top** : gibt das oberste Element des Stacks zurück, sofern es eines gibt, sonst 0
4. **pop** : entfernt das oberste Element des Stacks, sofern es eines gibt
5. **size** : liefert die Anzahl der Elemente im Stack

### 6.5 Algebraische Spezifikation: Eigenschaften von Algebren (4 Punkte)

Vervollständigen Sie:

1. Eine algebraische Spezifikation ist *redundant*, wenn ...
2. Eine algebraische Spezifikation ist *widersprüchlich*, wenn ...
3. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip *no-junk*, wenn ...
4. Eine Algebra folgt bzgl. einer algebraischen Spezifikation dem Prinzip *no-confusion*, wenn ...

### 6.6 Algebraische Spezifikation: Auswertung I (4 Punkte)

Gegeben sei die Spezifikation einer Liste, die insbesondere folgende Axiome enthält.

I  $\text{head}(\text{add}(L, e)) = \text{head}(L)$

II  $\text{head}(\text{add}(\text{new}(), e)) = e$

Zeigen Sie, wie sich mit den beiden Axiomen der Funktionswert  $\text{head}([1,2,3,4])$  berechnen lässt.

### 6.7 Algebraische Spezifikation: Auswertung II (5 Punkte)

Gegeben sei die Spezifikation einer Liste, die insbesondere folgende Axiome enthält.

$$\text{I } \text{con}(L, \text{new}()) = L$$

$$\text{II } \text{con}(L_1, \text{add}(L_2, e)) = \text{add}(\text{con}(L_1, L_2), e)$$

Zeigen Sie, dass sich die Gleichung (III)  $\text{con}(\text{new}(), L) = L$  aus den Axiomen ableiten lässt.

### 6.8 Kopplung und Kohäsion: Metrik (4 Punkte)

Überlegen Sie sich Metriken zur Messung von Kopplung und Kohäsion eines Software-Systems. Beschreiben Sie diese und veranschaulichen Sie sie an einem kleinen Beispiel.

### 6.9 Architekturstile: Beispiele (8 Punkte)

Beschreiben Sie für drei der folgenden Architekturstile Beispiele. Sie können auch ein System beschreiben, das mehrere Stile vereint.

1. Client-Server-Architektur
2. Objekt-Broker
3. Peer-to-peer-System
4. Ereignisgesteuertes System
5. Publish-subscribe
6. Schichten-Architektur
7. Repositories
8. Blackboard-System
9. Model-View-Controller

### 6.10 Entwurfsprinzipien: Beispiele (8 Punkte)

Beschreiben Sie für drei der folgenden Entwurfsprinzipien jeweils ein Beispiel:

1. Separation of Concerns
2. Single Responsibility Principle
3. Open-Closed Principle
4. Interface-Segregation Principle
5. Dependency Inversion Principle

6. Law of Demeter
7. Design by Contract
8. Don't Repeat Yourself
9. Self-documentation
10. Design for change